

Practical No - 8

Conclusion

The KNN algorithm was successfully implemented on the Iris dataset.

The model classified the data accurately using nearest neighbors and achieved **high accuracy**.

The performance of KNN depends on the value of **k** and distance calculation.

Practical Related Questions

1. How does k-NN work?

It finds the **k nearest data points** and predicts the class using **majority voting**.

2. What distance metrics are commonly used?

- Euclidean distance
 - Manhattan distance
 - Minkowski distance
-

3. What is the role of distance metrics in k-NN?

They calculate how close data points are to each other.

4. What is the effect of choosing different values of k?

- Small k → overfitting
 - Large k → underfitting
-

5. What are the drawbacks of k-NN?

- Slow for large datasets
 - Requires high memory
 - Sensitive to noisy data
-

6. How do you evaluate k-NN performance?

Using **accuracy score**, confusion matrix, precision, recall.

XII Exercise

1. Build KNN Classification model and compare results for different k values

```
# KNN Classification using Updated Sports Dataset
```

```
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
# Load dataset
data = pd.read_csv("sport.csv")
```

```
# Features & Target
X = data[["Age", "Fitness"]]
y = data["PlaysSports"]
```

```
# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42,
    stratify=y
)
```

```
# Try different K values
for k in [1, 3, 5]:
```

```
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
```

```
    y_pred = model.predict(X_test)
```

```
    acc = accuracy_score(y_test, y_pred)
```

```
    print("K =", k, "Accuracy =", acc)
```

Output

K Value	Accuracy	Observation
1	0.75	Overfitting possible
3	0.75	Best balance
5	0.625	More generalized

KNN was implemented using K = 1, 3, and 5.

K = 3 provided balanced performance, while higher K reduced accuracy.

Practical No - 9

X. Conclusion

Support Vector Machine (SVM) model was successfully implemented on the Iris dataset.

The model was trained using standardized features to improve performance.

SVM accurately classified the data by finding the optimal hyperplane with maximum margin.

The obtained accuracy shows that SVM is an effective and reliable algorithm for classification problems.

XI. Practical Related Questions

1. What is a Support Vector Machine (SVM)? Define kernel function.

Answer:

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression.

It works by finding the best separating boundary (hyperplane) between classes.

A **kernel function** transforms data into a higher-dimensional space so that non-linear data can be separated linearly.

Examples: Linear, Polynomial, RBF kernels.

2. Difference between Linear and Non-Linear SVM with example

Linear SVM	Non-Linear SVM
Used when data is linearly separable	Used when data is not linearly separable
Uses straight line / plane	Uses curves or complex boundaries
Example: Spam vs Not Spam	Example: Iris flower classification

3. Load any dataset and train an SVM model. Display accuracy.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = SVC(kernel='linear')
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

4. Compare Linear kernel vs RBF kernel. Which performs better and why?

Aspect	Linear Kernel	RBF Kernel
Data type	Works with linearly separable data	Works with non-linear data
Decision boundary	Straight line	Curved / complex
Flexibility	Less flexible	More flexible
Accuracy	Lower on complex data	Higher on complex data
When to use	Simple or large feature data	Real-world, complex datasets

RBF kernel performs better in most real-world problems because data is usually non-linear

XII. Exercise

Implement SVM using different kernels and compare accuracy

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

kernels = ['linear', 'poly', 'rbf']

for k in kernels:
    model = SVC(kernel=k)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    print("Kernel:", k, "Accuracy:", accuracy_score(y_test, y_pred))
```

Output

Kernel	Accuracy
Linear	0.96
Polynomial	0.96
RBF	1.00

Among all kernel functions, **RBF kernel gives the highest accuracy** because it effectively handles non-linear data.

Hence, RBF kernel is preferred for complex datasets.